

CS293 Project : Plagiarism Checker

Harshil Solanki
23B1016

Dhruvraj Merchant
23B1041

Kunj Bhesaniya
23B0995

November 24, 2024

1 Phase One: Barebones Checking of Two Submissions

1.1 Overview of the code:

1. `hashing` :

Computes the hash value of the first window of a given sequence using modular arithmetic.

l:

- Loops through the sequence (text) and calculates the hash by multiplying each character value with powers of a base (33 in this case), taking the modulo to prevent overflow.
- Returns the hash value and the last power of the base used.

Time Complexity: $O(len)$ where len is window length

Space Complexity: $O(1)$

2. `calculate_hashes` :

Generates hash values for all windows of size len in the input sequence using the rolling hash technique.

Working:

- Computes hashes for all windows using the rolling hash technique by updating the hash incrementally for each new window.
- Stores hash values and their starting indices in an unordered multimap (`hash_set`).

Time Complexity: $O(N)$ where N is length of text

Space Complexity: $O(N)$ due to hash storage

3. `rolling_hash` :

Finds short matching patterns of varying lengths between two sequences, remove overlapping matches, and calculate the total length of non-overlapping matches.

Working:

- For each pattern length (from 10 to 20), generate hash values for submission1 using `calculate_hashes`.
- Compare hash values of submission2 windows with those in the hash set of submission1.
- Create a Match object for each found match, store it, and remove the used hash to avoid duplicates.
- Call `remove_overlap` to eliminate overlaps and compute the total matching length.

Time Complexity: $O(N_1 + N_2 \log N_1 + M \log M)$, where N_1 and N_2 are size of token sequence & M = no. of matches found.

Space Complexity: $O(N + M)$, due to hash storage and match storage.

4. `upper_nonoverlap_match`

Finds the first non-overlapping match after a given match using binary search.

Working:

- Searches for the first match whose starting indices in both files (`start_file1` and `start_file2`) are greater than the current match's ending indices (`end_file1` and `end_file2`).
- Returns a pointer to the first non-overlapping match, or `nullptr` if no such match exists.

Time Complexity: $O(\log M)$ where M is number of matches

Space Complexity: $O(1)$

5. `remove_overlap` :

Removes overlapping matches and maximizes the total length of non-overlapping matches using dynamic programming.

Working:

- Matches are sorted based on their starting indices.

- For each match, use the `upper_nonoverlap_match` to find the first non-overlapping match. Update the DP array to store the maximum length of non-overlapping matches up to that point.
- Returns the maximum length of non-overlapping matches is stored in `dp[0]`.

Time Complexity: $O(M \log M)$ for sorting, $O(M \log M)$ for DP with binary search

Space Complexity: $O(M)$ for the DP array.

6. `similarity_score` :

Calculates the similarity score based on the given window size and matching length.

$$\text{similarity_score} = \frac{\text{matching_length}}{\text{window_size}}$$

7. `Large_pattern` :

Finds the largest pattern with a similarity score greater than a threshold by comparing pairs of elements in the LCS.

Working:

- For each pair of elements in lcs, it calculates the window size and matching length, if the similarity score meets the threshold, it updates the largest pattern found.

Time Complexity: $O(n^2)$, where n is the size of lcs.

8. `filter_lcs`:

Filters the LCS to keep only continuous sequences of at least 10 elements.

Working:

- Iterates through the LCS and checks for continuous subsequences.
- If the subsequence length is less than 10, the elements are marked for removal.
- After processing, removes elements with element = -1 from the lcs vector.

Time Complexity: $O(n)$, where n is the size of lcs.

Space Complexity: $O(1)$ (in-place modification of lcs).

9. `longestPattern`:

Finds the longest pattern match between two submissions using Dynamic Programming (DP) to calculate the Longest Common Subsequence (LCS).

Working:

- Constructs a DP table to find the LCS between sub1 and sub2.
- Traces back through the table to reconstruct the LCS.
- Filters the LCS to remove small patterns (less than 10 elements).
- Finds the largest pattern with a similarity score above the threshold.

Time Complexity: $O(mn)$, where m and n are the sizes of the two submissions.

Space Complexity: $O(mn)$

10. `evaluate_plag`:

Evaluates plagiarism based on a threshold: if the short match is at least 60% of the total length, or the large match is at least 50%, it returns true, indicating plagiarism.

Overall Time Complexity: $O(nm)$, where n and m are the lengths of the two submissions being compared.

Overall Space Complexity: $O(nm)$, due to the space used by the LCS computation and rolling hash storage.

2 Phase Two: Phase Two: A Full-Fledged Plagiarism Checker

2.1 Overview of the code:

1. `patch_work`

Detects patchwork plagiarism by checking for over matches spanning multiple submissions

Working:

- Sorts matches by end index.
- Counts unique matches and submissions involved.
- Flags as plagiarism if match count exceeds 20 and involves multiple submissions.

Time Complexity: $O(n \log n)$ for sorting matches intervals.

2. `flag_them`

Flags a submission (or two, if required) as plagiarized.

Working:

- Uses a mutex to safely update the flags.
 - Marks submissions as flagged and notifies the student and professor if required.
3. **handle_and_merge_matches:**
Processes and merges continuous matches between two submissions.
Working:
- Checks for continuity between matches.
 - If continuous, extends the current match; otherwise, adds a new match to the stack.
 - Marks tokens as matched to avoid duplication.
4. **handle_small_match:**
Handles smaller matches (less than 15 tokens) and merges them with continuous matches.
Working:
- Tries to extend the current match by checking smaller matches.
 - Updates indices and continuous match size.
5. **check_plagiarism:**
Compares two submissions to detect plagiarism.
Working:
- Calculates hashes of one submission (submission2).
 - Performs a rolling hash comparison with the other (submission1).
 - Tracks matches using a stack, handles continuous and small matches, and flags submissions if thresholds (like match count) are exceeded.
 - Returns all matches for further processing (e.g., patchwork plagiarism).
- Time Complexity: $O(n + m)$ Where n and m are the token counts of the two submissions.
6. **process_plagcheck_for_submission:**
Processes plagiarism checks for a single submission against all others.
Working:
- Iterates over all submissions.
 - Skips the submission itself.
 - Calls check_plagiarism for each comparison.
 - Checks for patchwork plagiarism in the end.
- Time Complexity: $O(s(n + m))$: Where s is the number of other submissions and n, m are the token counts.
7. **add_submission**
Adds a new submission and queues its plagiarism check.
Working:
- Stores the submission and tokens.
 - Enqueues the plagiarism check using the thread pool.
8. **~plagiarism_checker_t():**
Ensures all threads in the thread pool complete before the object is destroyed.
Working:
- Calls join_threads on the thread pool to block until all threads finish.
9. **ThreadPool Class:**
The ThreadPool class provides a mechanism to manage and execute multiple tasks concurrently using a fixed pool of worker threads. It allows tasks to be enqueued and executed asynchronously, improving the efficiency of programs that can leverage parallelism.
Working:
- **enqueue(F f)**
Allows a user to add tasks to the thread pool for execution.
Working:
 - Adds the task to the queue and notifies a worker thread to pick it up.
 - **join_threads():**
shuts down the thread pool.
Working:
 - Signals all threads to stop after completing their current tasks.
 - Waits for all threads to finish execution using std::thread::join.